CSA4604 — MERN Stack Lab

Observation Notes — Experiments 10 to 19

Experiment 10: CRUD Operations and Update Queries on Student Database using MongoDB

Aim: To perform CRUD operations and update queries on a student database using MongoDB.

Observations:

- Started the MongoDB server and opened `mongosh`.
- Created a database `school` and a collection `students` using `use school`.
- Inserted one and many documents using `insertOne()` and `insertMany()`.
- Read documents using `find()` and `findOne()` with filters and query operators (`$gt`, `$gte`, `$lt`, `$ne`, `$in`, `$or`).
- Updated documents using `updateOne()` and `updateMany()` with `$set`, `$unset`, `$inc` and `$push`.
- Used `upsert: true` to insert a document when no match was found.
- Deleted documents using `deleteOne()` and `deleteMany()`.

Commands used:

```
use school

// Create
db.students.insertOne({ name: 'Alex', age: 21, grade: 'A' })
db.students.insertMany([
  { name: 'Ravi', age: 22, grade: 'B' },
  { name: 'Sara', age: 20, grade: 'A' },
  { name: 'Mina', age: 23, grade: 'C' }
])

// Read
db.students.find()
db.students.find({ grade: 'A' })
db.students.find({ age: { $gt: 20 } })
db.students.find({ grade: { $in: ['A', 'B'] } })
db.students.find({ $or: [{ grade: 'C' }, { age: { $lt: 21 } }] })
db.students.findOne({ name: 'Sara' })

// Update
db.students.updateOne({ name: 'Ravi' }, { $set: { grade: 'A' } })
db.students.updateMany({}, { $inc: { age: 1 } })
db.students.updateOne({ name: 'Alex' }, { $push: { skills: 'React' } })
db.students.updateOne({ name: 'Alex' }, { $unset: { skills: '' } })
db.students.updateOne(
  { name: 'Tina' },
  { $set: { age: 19, grade: 'B' } },
  { upsert: true }
)

// Delete
```

```
db.students.deleteOne({ name: 'Mina' })
db.students.deleteMany({ grade: 'C' })
```

Experiment 11: Query Modifiers and Aggregation Operations on Student Database

Aim: To use query modifiers and the aggregation framework on a student database in MongoDB.

Observations:

- Created a database `campus` with a `students` collection of 12 documents (`name`, `dept`, `city`, `marks`, `attendance`).
- Used projection to return selected fields and hide `_id`.
- Used `sort()`, `limit()` and `skip()` to order, restrict and paginate results.
- Used `countDocuments()` and `distinct()` for counts and unique values.
- Built aggregation pipelines with `$match`, `$group`, `$sort` and `$project`.
- Used the accumulators `$sum`, `$avg`, `$max` and `$min` inside `$group`.
- Observed that the order of pipeline stages changes the result.

Commands used:

```
use campus

db.students.insertMany([
  { name: 'Asha',  dept: 'CSE', city: 'Bengaluru', marks: 88, attendance: 92 },
  { name: 'Bala',  dept: 'CSE', city: 'Mysuru',    marks: 74, attendance: 85 },
  { name: 'Chitra', dept: 'ECE', city: 'Bengaluru', marks: 91, attendance: 95 },
  { name: 'Dev',   dept: 'ECE', city: 'Hubballi',  marks: 67, attendance: 78 },
  { name: 'Esha',  dept: 'MBA', city: 'Bengaluru', marks: 82, attendance: 90 },
  { name: 'Farhan', dept: 'MBA', city: 'Mysuru',   marks: 59, attendance: 70 }
])

// Query modifiers
db.students.find({}, { name: 1, marks: 1, _id: 0 })
db.students.find().sort({ marks: -1 }).limit(3)
db.students.find().sort({ name: 1 }).skip(2).limit(2)
db.students.countDocuments({ dept: 'CSE' })
db.students.distinct('city')

// Aggregation
db.students.aggregate([
  { $group: { _id: '$dept', total: { $sum: 1 } } }
])

db.students.aggregate([
  { $group: { _id: '$dept', avgMarks: { $avg: '$marks' } } },
  { $sort: { avgMarks: -1 } }
])

db.students.aggregate([
  { $match: { city: 'Bengaluru' } },
  { $group: { _id: '$dept', avgMarks: { $avg: '$marks' } } }
])

db.students.aggregate([
```

```
  { $group: {
      _id: '$dept',
      best: { $max: '$marks' },
      worst: { $min: '$marks' },
      avg: { $avg: '$marks' }
  } },
  { $project: { _id: 0, dept: '$_id', best: 1, worst: 1, avg: 1 } }
])
```

Experiment 12: CRUD Operations and Update Queries on Employee Information Database

Aim: To perform CRUD operations and update queries on an employee information database using MongoDB.

Observations:

- Created a database `company` with an `employees` collection.
- Inserted employee documents with nested (`address`) and array (`skills`) fields.
- Queried nested fields using dot notation and array fields using `$in` and `$all`.
- Updated salary using `$inc` and `$mul`, and renamed a field using `$rename`.
- Added and removed array items using `$push`, `$addToSet` and `$pull`.
- Verified that `$addToSet` ignores duplicates while `$push` does not.
- Deleted employees by condition.
- Created an index on `dept` and compared the query plan using `explain()`.

Commands used:

```
use company

db.employees.insertMany([
  { name: 'Kiran', dept: 'IT', designation: 'Developer', salary: 55000,
    skills: ['Node', 'React'], address: { city: 'Bengaluru', pin: 560064 } },
  { name: 'Meera', dept: 'HR', designation: 'Manager', salary: 62000,
    skills: ['Hiring'], address: { city: 'Mysuru', pin: 570001 } },
  { name: 'Nikhil', dept: 'IT', designation: 'Tester', salary: 42000,
    skills: ['Selenium', 'Node'], address: { city: 'Bengaluru', pin: 560064 } }
])

// Read
db.employees.find({ 'address.city': 'Bengaluru' })
db.employees.find({ skills: { $in: ['Node'] } })
db.employees.find({ skills: { $all: ['Node', 'React'] } })
db.employees.find({ salary: { $gte: 50000 } }, { name: 1, salary: 1, _id: 0 })

// Update
db.employees.updateOne({ name: 'Kiran' }, { $inc: { salary: 5000 } })
db.employees.updateMany({ dept: 'IT' }, { $mul: { salary: 1.1 } })
db.employees.updateOne({ name: 'Meera' }, { $rename: { designation: 'role' } })
db.employees.updateOne({ name: 'Kiran' }, { $push: { skills: 'MongoDB' } })
db.employees.updateOne({ name: 'Kiran' }, { $addToSet: { skills: 'MongoDB' } })
db.employees.updateOne({ name: 'Kiran' }, { $pull: { skills: 'React' } })

// Delete
db.employees.deleteOne({ name: 'Nikhil' })
```

```
db.employees.deleteMany({ salary: { $lt: 30000 } })

// Index
db.employees.createIndex({ dept: 1 })
db.employees.find({ dept: 'IT' }).explain('executionStats')
db.employees.getIndexes()
```

Experiment 13: Connecting a Node.js Application with MongoDB using MongoDB Driver and Mongoose

Aim: To connect a Node.js application with MongoDB using the MongoDB driver and Mongoose, and perform CRUD operations.

Observations:

- Created a Node.js project and enabled ES6 modules with `"type": "module"`.
- Installed the packages using `npm install mongodb mongoose dotenv`.
- Stored the connection address in a `.env` file and read it using `import 'dotenv/config'`.
- Connected to MongoDB using `MongoClient` and `await client.connect()`, then performed CRUD directly on a collection.
- Connected using `mongoose.connect()`, defined a schema and created a model.
- Observed that the model file does not connect to the database; the connection is opened once in the calling script.
- Observed that Mongoose validates data against the schema (required fields, allowed values, range) before saving.
- Disconnected at the end of each script so that it could finish.

Commands used:

```
npm init -y
npm install mongodb mongoose dotenv
```

Code:

```
# .env
MONGO_URI=mongodb://127.0.0.1:27017/school
```

```
// driver.js — MongoDB native driver
import 'dotenv/config';
import { MongoClient } from 'mongodb';

const client = new MongoClient(process.env.MONGO_URI);
await client.connect();

const students = client.db('school').collection('students');

await students.insertOne({ name: 'Alex', age: 21, grade: 'A' });
console.log(await students.find({ grade: 'A' }).toArray());
await students.updateOne({ name: 'Alex' }, { $set: { age: 22 } });
await students.deleteOne({ name: 'Alex' });
```

```
await client.close();
```

```
// models/Student.js — Mongoose schema and model
import mongoose from 'mongoose';

const studentSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  age: { type: Number, min: 5, max: 100 },
  grade: { type: String, enum: ['A', 'B', 'C', 'D'], required: true }
});

export default mongoose.model('Student', studentSchema);
```

```
// mongoose-crud.js
import 'dotenv/config';
import mongoose from 'mongoose';
import Student from './models/Student.js';

await mongoose.connect(process.env.MONGO_URI);

// Create
await Student.create({ name: 'Ravi', age: 22, grade: 'B' });
await Student.insertMany([
  { name: 'Sara', age: 20, grade: 'A' },
  { name: 'Mina', age: 23, grade: 'C' }
]);

// Read
console.log(await Student.find());
console.log(await Student.find({ age: { $gt: 20 } }).sort({ age: -1 }).limit(2));
const sara = await Student.findOne({ name: 'Sara' });
console.log(await Student.findById(sara._id));

// Update
await Student.findByIdAndUpdate(
  sara._id,
  { grade: 'B' },
  { returnDocument: 'after', runValidators: true }
);
await Student.updateMany({ grade: 'C' }, { $inc: { age: 1 } });

// Delete
await Student.findByIdAndDelete(sara._id);
await Student.deleteMany({ grade: 'C' });

await mongoose.disconnect();
```

Experiment 14: Exploring npm Commands and Installing External Modules using npm

Aim: To explore commonly used npm commands and install and use external modules in a Node.js application.

Observations:

- Initialized a project using `npm init -y` and inspected `package.json`.
- Installed packages as dependencies and as development dependencies (`-D`).
- Observed the `node_modules` folder, `package-lock.json` and the `^` version prefix.
- Listed installed packages using `npm list` and checked for newer versions using `npm outdated`.
- Updated and uninstalled packages using `npm update` and `npm uninstall`.
- Added custom scripts (`start`, `dev`) and ran them using `npm run`.
- Ran a package without installing it using `npx`.
- Used the external modules `lodash`, `dayjs` and `chalk` in an ES6 program.
- Rebuilt `node_modules` from `package.json` using `npm install`, which is why `node_modules` is not pushed to GitHub (added to `.gitignore`).

Commands used:

```
npm init -y
npm install lodash dayjs chalk
npm install -D nodemon
npm list --depth=0
npm outdated
npm update lodash
npm uninstall dayjs
npm run dev
npx cowsay "Hello MERN"
npm install            # restores node_modules from package.json
```

Code:

```
// package.json (relevant part)
{
  "type": "module",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  }
}
```

```
// index.js
import _ from 'lodash';
import dayjs from 'dayjs';
import chalk from 'chalk';

console.log(_.chunk([1, 2, 3, 4, 5], 2));
console.log(_.capitalize('mern stack'));
console.log(dayjs().format('DD MMM YYYY, HH:mm'));
console.log(chalk.green('npm module working'));
```

Experiment 15: Developing React Applications using JSX, Components and Props

Aim: To develop a React application using JSX, functional components and props.

Observations:

- Created a React project using Vite and started the development server.
- Understood the project structure (`index.html`, `src/main.jsx`, `src/App.jsx`).
- Wrote JSX, which looks like HTML but is converted to JavaScript.
- Observed the JSX rules: a single parent element, `className` instead of `class`, and JavaScript expressions inside `{ }`.
- Created reusable functional components, each starting with a capital letter.
- Passed data from parent to child using props and destructured them.
- Rendered a list from an array using `map()` with a unique `key`.
- Observed that props are read-only.

Commands used:

```
npm create vite@latest react-lab -- --template react
npm install
npm run dev
```

Code:

```
// src/StudentCard.jsx
export default function StudentCard({ name, grade, age = 20 }) {
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>Grade: {grade}</p>
      <p>Age: {age}</p>
    </div>
  );
}
```

```
// src/App.jsx
import StudentCard from './StudentCard.jsx';

const students = [
  { id: 1, name: 'Alex', grade: 'A', age: 21 },
  { id: 2, name: 'Ravi', grade: 'B' },
  { id: 3, name: 'Sara', grade: 'A', age: 22 }
];

export default function App() {
  return (
    <div>
      <h1>Student List</h1>
      {students.map((s) => (
        <StudentCard key={s.id} name={s.name} grade={s.grade} age={s.age} />
      ))}
    </div>
```

```
  );
}
```

Experiment 16: Developing React Applications: State Management, Event Handling and Component Communication

Aim: To manage state, handle events and enable communication between components in a React application.

Observations:

- Used the `useState` hook to store data that changes over time.
- Observed that calling the setter function re-renders the component, while changing a normal variable does not.
- Handled `onClick` and `onChange` events and read values from `event.target.value`.
- Built a controlled input whose value is held in state.
- Handled form submission using `onSubmit` with `event.preventDefault()`.
- Updated array state without mutation using spread (`[...items, newItem]`) and `filter()`.
- Achieved child-to-parent communication by passing a function as a prop.
- Observed that state is shared by lifting it up to the nearest common parent.

Code:

```
// src/Counter.jsx
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

```
// src/NameForm.jsx — child sends data to parent through a prop
import { useState } from 'react';

export default function NameForm({ onAdd }) {
  const [name, setName] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!name.trim()) return;
    onAdd(name);
    setName('');
  };

  return (
    <form onSubmit={handleSubmit}>
```

```
      <input value={name} onChange={(e) => setName(e.target.value)} placeholder="Student name"
/>
      <button type="submit">Add</button>
    </form>
  );
}
```

```
// src/App.jsx — parent holds the shared state
import { useState } from 'react';
import Counter from './Counter.jsx';
import NameForm from './NameForm.jsx';

export default function App() {
  const [students, setStudents] = useState(['Alex', 'Ravi']);

  const addStudent = (name) => setStudents([...students, name]);
  const removeStudent = (name) => setStudents(students.filter((s) => s !== name));

  return (
    <div>
      <Counter />
      <NameForm onAdd={addStudent} />
      <ul>
        {students.map((s) => (
          <li key={s}>
            {s} <button onClick={() => removeStudent(s)}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Experiment 17: Designing Responsive User Interface Components using Bootstrap in React Applications

Aim: To design responsive user interface components in a React application using Bootstrap.

Observations:

- Installed Bootstrap using npm and imported its CSS in `main.jsx`.
- Used the Bootstrap grid (`container`, `row`, `col-*`) to build a layout that adapts to screen size.
- Observed the breakpoints (`sm`, `md`, `lg`) by resizing the browser window.
- Built a navbar, cards, buttons and a form using Bootstrap classes.
- Used spacing utilities (`p-3`, `mt-4`, `gap-2`) and responsive columns (`col-12 col-md-6 col-lg-4`).
- Observed that three cards in a row on a large screen became one column on a mobile screen.
- Used `className` instead of `class` for Bootstrap classes in JSX.

Commands used:

```
npm install bootstrap
```

Code:

```
// src/main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import 'bootstrap/dist/css/bootstrap.min.css';
import App from './App.jsx';

ReactDOM.createRoot(document.getElementById('root')).render(<App />);
```

```
// src/App.jsx
const courses = [
  { id: 1, title: 'Node.js', text: 'Server-side JavaScript' },
  { id: 2, title: 'MongoDB', text: 'NoSQL database' },
  { id: 3, title: 'React', text: 'Frontend library' }
];

export default function App() {
  return (
    <>
      <nav className="navbar navbar-dark bg-dark px-3">
        <span className="navbar-brand">MERN Lab</span>
      </nav>

      <div className="container mt-4">
        <div className="row g-3">
          {courses.map((c) => (
            <div key={c.id} className="col-12 col-md-6 col-lg-4">
              <div className="card h-100 shadow-sm">
                <div className="card-body">
                  <h5 className="card-title">{c.title}</h5>
                  <p className="card-text">{c.text}</p>
                  <button className="btn btn-primary">Learn more</button>
                </div>
              </div>
            </div>
          ))}
        </div>

        <form className="mt-4 p-3 border rounded">
          <div className="mb-3">
            <label className="form-label">Email</label>
            <input type="email" className="form-control" />
          </div>
          <button type="submit" className="btn btn-success w-100">Subscribe</button>
        </form>
      </div>
    </>
  );
}
```

Experiment 18: Development of a React-Based Food Delivery Website Application

Aim: To develop a food delivery website using React components, state and Bootstrap.

Observations:

- Created a React project and installed Bootstrap.
- Stored the menu as an array of food items (name, price, category).
- Split the page into components: `Navbar`, `FoodCard` and `Cart`.
- Used state in `App` to hold the cart and the selected category.
- Filtered the menu by category using `filter()`.
- Added items to the cart, increased the quantity of an item already in the cart, and removed items.
- Calculated the cart total using `reduce()`.
- Observed that the layout stayed responsive on small screens.
- Observed that the cart resets on page refresh, as the data is held in state only.

Commands used:

```
npm create vite@latest food-delivery -- --template react
npm install
npm install bootstrap
npm run dev
```

Code:

```
// src/data.js
export const menu = [
  { id: 1, name: 'Margherita Pizza', price: 249, category: 'Pizza' },
  { id: 2, name: 'Veg Burger', price: 129, category: 'Burger' },
  { id: 3, name: 'Paneer Wrap', price: 149, category: 'Wrap' },
  { id: 4, name: 'Cheese Pizza', price: 299, category: 'Pizza' },
  { id: 5, name: 'Chicken Burger', price: 179, category: 'Burger' }
];
```

```
// src/FoodCard.jsx
export default function FoodCard({ item, onAdd }) {
  return (
    <div className="card h-100 shadow-sm">
      <div className="card-body">
        <h5>{item.name}</h5>
        <p className="text-muted">{item.category}</p>
        <p className="fw-bold">₹{item.price}</p>
        <button className="btn btn-primary" onClick={() => onAdd(item)}>Add to cart</button>
      </div>
    </div>
  );
}
```

```
// src/Cart.jsx
```

```
export default function Cart({ cart, onRemove }) {
  const total = cart.reduce((sum, i) => sum + i.price * i.qty, 0);

  return (
    <div className="card p-3">
      <h5>Your Cart</h5>
      {cart.length === 0 && <p className="text-muted">Cart is empty</p>}
      {cart.map((i) => (
        <div key={i.id} className="d-flex justify-content-between mb-2">
          <span>{i.name} × {i.qty}</span>
          <span>
            ₹{i.price * i.qty}{' '}
            <button className="btn btn-sm btn-outline-danger" onClick={() =>
onRemove(i.id)}>x</button>
          </span>
        </div>
      ))}
      <hr />
      <h6>Total: ₹{total}</h6>
    </div>
  );
}
```

```
// src/App.jsx
import { useState } from 'react';
import { menu } from './data.js';
import FoodCard from './FoodCard.jsx';
import Cart from './Cart.jsx';

const categories = ['All', ...new Set(menu.map((m) => m.category))];

export default function App() {
  const [cart, setCart] = useState([]);
  const [category, setCategory] = useState('All');

  const addToCart = (item) => {
    const exists = cart.find((c) => c.id === item.id);
    if (exists) {
      setCart(cart.map((c) => (c.id === item.id ? { ...c, qty: c.qty + 1 } : c)));
    } else {
      setCart([...cart, { ...item, qty: 1 }]);
    }
  };

  const removeFromCart = (id) => setCart(cart.filter((c) => c.id !== id));

  const visible = category === 'All' ? menu : menu.filter((m) => m.category === category);

  return (
    <>
      <nav className="navbar navbar-dark bg-danger px-3">
        <span className="navbar-brand">FoodGo</span>
        <span className="text-white">Cart: {cart.reduce((n, c) => n + c.qty, 0)}</span>
      </nav>

      <div className="container mt-4">
        <div className="mb-3 d-flex gap-2 flex-wrap">
          {categories.map((c) => (
            <button
              key={c}
              className={`btn ${category === c ? 'btn-dark' : 'btn-outline-dark'}`}
              onClick={() => setCategory(c)}
            >
              {c}
            </button>
```

```
          ))}
        </div>

        <div className="row g-3">
          <div className="col-12 col-lg-8">
            <div className="row g-3">
              {visible.map((item) => (
                <div key={item.id} className="col-12 col-sm-6">
                  <FoodCard item={item} onAdd={addToCart} />
                </div>
              ))}
            </div>
          </div>
          <div className="col-12 col-lg-4">
            <Cart cart={cart} onRemove={removeFromCart} />
          </div>
        </div>
      </div>
    </>
  );
}
```

Experiment 19: Development of a MERN Stack To-Do List Application with CRUD Operations and User Authentication

Aim: To develop a full-stack To-Do List application with CRUD operations and user authentication using MongoDB, Express.js, React and Node.js.

Observations:

- Created two folders: `server` (Express + MongoDB) and `client` (React).
- Stored the connection string and the JWT secret in a `.env` file.
- Created `User` and `Todo` models using Mongoose.
- Stored passwords after hashing them with `bcryptjs`; plain-text passwords were never saved.
- Implemented `register` and `login` routes; login returned a signed JWT.
- Wrote an `auth` middleware that verified the token from the `Authorization` header and rejected invalid requests with status 401.
- Implemented GET, POST, PUT and DELETE routes for to-dos behind the `auth` middleware.
- Observed that each user could see and change only their own to-dos, as every query filtered by the logged-in user's id.
- Used `cors()` so that the React app (port 5173) could call the API (port 5000).
- Built the React interface with `useState` and `useEffect`, stored the token in `localStorage` and sent it with each request.
- Tested the flow: register, login, add, tick as done, edit, delete, logout. Data persisted after restarting the server.

Commands used:

```
# Backend (server folder)
npm init -y
npm install express mongoose cors dotenv bcryptjs jsonwebtoken
npm install -D nodemon

# Frontend (client folder)
npm create vite@latest client -- --template react
npm install
npm install bootstrap
npm run dev
```

Code:

```
# server/.env
MONGO_URI=mongodb://127.0.0.1:27017/todoapp
JWT_SECRET=change_this_secret
PORT=5000
```

```
// server/index.js
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import mongoose from 'mongoose';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';

const app = express();
app.use(cors());
app.use(express.json());

await mongoose.connect(process.env.MONGO_URI);

// Models
const User = mongoose.model('User', new mongoose.Schema({
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }
}));

const Todo = mongoose.model('Todo', new mongoose.Schema({
  text: { type: String, required: true },
  done: { type: Boolean, default: false },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
}));

// Auth middleware
const auth = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  try {
    req.userId = jwt.verify(token, process.env.JWT_SECRET).id;
    next();
  } catch {
    res.status(401).send({ message: 'Unauthorized' });
  }
};

// Auth routes
app.post('/api/auth/register', async (req, res) => {
  const { email, password } = req.body;
  if (await User.findOne({ email })) {
    return res.status(400).send({ message: 'User already exists' });
  }
```

```
  const hashed = await bcrypt.hash(password, 10);
  await User.create({ email, password: hashed });
  res.status(201).send({ message: 'Registered' });
});

app.post('/api/auth/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.status(400).send({ message: 'Invalid email or password' });
  }
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1d' });
  res.send({ token });
});

// Todo routes (protected)
app.get('/api/todos', auth, async (req, res) => {
  res.send(await Todo.find({ user: req.userId }));
});

app.post('/api/todos', auth, async (req, res) => {
  const todo = await Todo.create({ text: req.body.text, user: req.userId });
  res.status(201).send(todo);
});

app.put('/api/todos/:id', auth, async (req, res) => {
  const todo = await Todo.findOneAndUpdate(
    { _id: req.params.id, user: req.userId },
    { text: req.body.text, done: req.body.done },
    { returnDocument: 'after', runValidators: true }
  );
  if (!todo) return res.status(404).send({ message: 'Todo not found' });
  res.send(todo);
});

app.delete('/api/todos/:id', auth, async (req, res) => {
  const todo = await Todo.findOneAndDelete({ _id: req.params.id, user: req.userId });
  if (!todo) return res.status(404).send({ message: 'Todo not found' });
  res.send(todo);
});

app.listen(process.env.PORT, () => {
  console.log(`Server running on port ${process.env.PORT}`);
});
```

```
// server/package.json (relevant part)
{
  "type": "module",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  }
}
```

```
// client/src/api.js
const API = 'http://localhost:5000/api';

export const api = async (path, method = 'GET', body) => {
  const token = localStorage.getItem('token');
  const res = await fetch(`${API}${path}`, {
    method,
    headers: {
      'Content-Type': 'application/json',
      ...(token && { Authorization: `Bearer ${token}` })
```

```
    },
    body: body && JSON.stringify(body)
  });
  const data = await res.json();
  if (!res.ok) throw new Error(data.message);
  return data;
};
```

```
// client/src/Auth.jsx
import { useState } from 'react';
import { api } from './api.js';

export default function Auth({ onLogin }) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [isLogin, setIsLogin] = useState(true);
  const [error, setError] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      if (isLogin) {
        const { token } = await api('/auth/login', 'POST', { email, password });
        localStorage.setItem('token', token);
        onLogin();
      } else {
        await api('/auth/register', 'POST', { email, password });
        setIsLogin(true);
        setError('Registered. Please login.');
      }
    } catch (err) {
      setError(err.message);
    }
  };

  return (
    <form className="container mt-5" style={{ maxWidth: 400 }} onSubmit={handleSubmit}>
      <h3>{isLogin ? 'Login' : 'Register'}</h3>
      {error && <div className="alert alert-info">{error}</div>}
      <input className="form-control mb-2" type="email" placeholder="Email"
        value={email} onChange={(e) => setEmail(e.target.value)} />
      <input className="form-control mb-2" type="password" placeholder="Password"
        value={password} onChange={(e) => setPassword(e.target.value)} />
      <button className="btn btn-primary w-100">{isLogin ? 'Login' : 'Register'}</button>
      <button type="button" className="btn btn-link w-100" onClick={() =>
setIsLogin(!isLogin)}>
        {isLogin ? 'Create an account' : 'Back to login'}
      </button>
    </form>
  );
}
```

```
// client/src/Todos.jsx
import { useState, useEffect } from 'react';
import { api } from './api.js';

export default function Todos({ onLogout }) {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState('');

  useEffect(() => {
    api('/todos').then(setTodos).catch(onLogout);
  }, []);
```

```
  const addTodo = async (e) => {
    e.preventDefault();
    if (!text.trim()) return;
    const todo = await api('/todos', 'POST', { text });
    setTodos([...todos, todo]);
    setText('');
  };

  const toggleTodo = async (todo) => {
    const updated = await api(`/todos/${todo._id}`, 'PUT', { text: todo.text, done: !todo.done
});
    setTodos(todos.map((t) => (t._id === todo._id ? updated : t)));
  };

  const deleteTodo = async (id) => {
    await api(`/todos/${id}`, 'DELETE');
    setTodos(todos.filter((t) => t._id !== id));
  };

  return (
    <div className="container mt-4" style={{ maxWidth: 500 }}>
      <div className="d-flex justify-content-between mb-3">
        <h3>My To-Dos</h3>
        <button className="btn btn-outline-secondary btn-sm" onClick={onLogout}>Logout</button>
      </div>

      <form className="d-flex gap-2 mb-3" onSubmit={addTodo}>
        <input className="form-control" placeholder="New task"
          value={text} onChange={(e) => setText(e.target.value)} />
        <button className="btn btn-primary">Add</button>
      </form>

      <ul className="list-group">
        {todos.map((t) => (
          <li key={t._id} className="list-group-item d-flex justify-content-between
align-items-center">
            <span
              style={{ textDecoration: t.done ? 'line-through' : 'none', cursor: 'pointer' }}
              onClick={() => toggleTodo(t)}
            >
              {t.text}
            </span>
            <button className="btn btn-sm btn-outline-danger" onClick={() =>
deleteTodo(t._id)}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```
// client/src/App.jsx
import { useState } from 'react';
import Auth from './Auth.jsx';
import Todos from './Todos.jsx';

export default function App() {
  const [loggedIn, setLoggedIn] = useState(!!localStorage.getItem('token'));

  const logout = () => {
    localStorage.removeItem('token');
    setLoggedIn(false);
  };

  return loggedIn
```

```
    ? <Todos onLogout={logout} />
    : <Auth onLogin={() => setLoggedIn(true)} />;
}
```

```
// client/src/main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import 'bootstrap/dist/css/bootstrap.min.css';
import App from './App.jsx';

ReactDOM.createRoot(document.getElementById('root')).render(<App />);
```